**Tribhuvan University**
Institute of Engineering
**Pulchowk Campus**

*A Project Report on*

# Real-Time and Historical Analytics of the Nepal Stock Exchange (NEPSE) using a Big Data Stack

*Apache Kafka · Apache Spark (Structured Streaming, SQL, MLlib) · Hadoop HDFS*

Submitted for the course **Big Data Technology**

**Submitted by**

Santosh Tiwari (082MSDSA024)
Arbind Sah (082MSDSA006)

**Submitted to**

Basanta Joshi

11 October 2026

# Abstract

Retail investors in Nepal follow the Nepal Stock Exchange (NEPSE) through several separate websites that show current prices but offer no combined view of real-time activity and long-term history. This project builds a big data system that collects NEPSE market data, processes it both as a live stream and as a 31-year historical batch, stores it in a Hadoop data lake and presents the results on an interactive dashboard.

The system uses **Apache Kafka** for ingestion and **Apache Spark** for processing, with **HDFS** (Parquet files) as the storage layer, following a Lambda-style architecture. Live prices come from the official NEPSE API; when the market is closed, a replay producer streams any past trading day through the same pipeline, so the demonstration never depends on market hours. The batch layer cleans 514,018 historical price records for 372 companies (repairing 40,552 inconsistent rows and correcting 731 bonus/right-share price adjustments) and computes technical indicators such as moving averages, RSI, Bollinger bands and 52-week ranges. The streaming layer produces 1-minute candles, sector money flow, a live market snapshot and alerts for circuit hits, volume spikes and large price moves. A fault-tolerance test in which the streaming job was killed mid-session showed no data loss and no duplication (15,142 of 15,142 ticks stored exactly once). As a bonus, a Spark MLlib random forest predicts next-day price direction; its honest result (55.8 % accuracy against a 55.9 % baseline) illustrates how close daily price moves are to random.

**Keywords:** big data, Apache Kafka, Apache Spark, Structured Streaming, HDFS, Parquet, Lambda architecture, NEPSE, stock market analytics.

# Table of Contents

**Abstract** ..... **ii**

**List of Figures** ..... **v**

**List of Tables** ..... **vi**

**List of Abbreviations** ..... **vii**

**1. Introduction** ..... **1**

1.1 Background ..... 1

1.2 Motivation ..... 1

1.3 Objectives ..... 1

1.4 Scope ..... 2

**2. Problem Definition** ..... **3**

2.1 Problem statement ..... 3

2.2 Why this is a big data problem ..... 3

**3. Data Sources** ..... **4**

3.1 Common message format ..... 4

**4. Important Features Considered** ..... **6**

4.1 Data features (raw) ..... 6

4.2 Derived analytical features ..... 6

4.3 System design features ..... 7

**5. System Design** ..... **8**

5.1 Choice of big data components ..... 8

5.2 Block diagram ..... 10

5.3 Data storage design ..... 11

5.4 Operating modes ..... 13

**6. Implementation** ..... **14**

6.1 Environment and technology stack ..... 14

6.2 Data collection and ingestion ..... 14

6.3 Batch processing (Spark SQL) ..... 15

6.4 Stream processing (Spark Structured Streaming) ..... 15

6.5 Machine learning (Spark MLlib, bonus) ..... 16

6.6 Dashboard ..... 16

**7. Results and Evaluation ..... 21**

7.1 Data quality ..... 21

7.2 Sample analytical results ..... 21

7.3 Streaming results ..... 21

7.4 Fault-tolerance test ..... 22

7.5 Performance and resources ..... 22

7.6 Machine-learning results ..... 22

7.7 Problems encountered and solutions ..... 23

**8. Demonstration Plan ..... 25**

**9. Limitations and Future Work ..... 26**

9.1 Limitations ..... 26

9.2 Future work ..... 26

**10. Conclusion ..... 27**

**References ..... 28**

**Appendix A: Installation and Execution ..... 29**

**Appendix B: Project Structure ..... 30**

**Appendix C: Division of Work ..... 31**

# List of Figures

Figure 1: Block diagram of the NEPSE big data system (speed layer above, batch layer below) ..... 10

Figure 2: Live market tab during a REPLAY session: mode badge, market KPIs, movers, sector map and Spark alerts ..... 17

Figure 3: Stock explorer: bonus-adjusted candlesticks with SMA 20/50/200, Bollinger band, volume and RSI ..... 18

Figure 4: Market & sectors tab: NEPSE index with moving averages, yearly returns and turnover by sector ..... 19

Figure 5: Screener: all securities filtered by sector, trend, RSI signal and volume spike ..... 19

Figure 6: Pipeline tab: architecture, Kafka topic sizes, data-quality report and HDFS datasets ..... 20

Figure 7: MLlib results and feature importance shown on the dashboard ..... 23

# List of Tables

Table 1: The five Vs of big data applied to NEPSE ..... 3

Table 2: Data sources ..... 4

Table 3: Raw data features ..... 6

Table 4: Derived features ..... 6

Table 5: Big data components chosen and alternatives ..... 8

Table 6: HDFS data lake layout ..... 11

Table 7: Kafka topics ..... 12

Table 8: Operating modes ..... 13

Table 9: Technology stack ..... 14

Table 10: Streaming queries ..... 15

Table 11: Data-quality report produced by the batch layer ..... 21

Table 12: NEPSE yearly returns computed with Spark SQL (selected years) ..... 21

Table 13: Performance measurements ..... 22

Table 14: Spark MLlib random forest — next-day direction ..... 22

Table 15: Problems encountered and how they were solved ..... 23

Table 16: Setup and execution commands ..... 29

Table 17: Web interfaces ..... 29

Table 18: Division of work ..... 31

# List of Abbreviations

| Abbreviation | Meaning |
|---|---|
| API | Application Programming Interface |
| AUC | Area Under the (ROC) Curve |
| CSV | Comma-Separated Values |
| HDFS | Hadoop Distributed File System |
| JSON / JSONL | JavaScript Object Notation / JSON Lines (one JSON object per line) |
| ML / MLlib | Machine Learning / Spark's Machine Learning Library |
| NEPSE | Nepal Stock Exchange |
| NPT | Nepal Time (UTC+05:45) |
| OHLC / OHLCV | Open, High, Low, Close (and Volume) prices of a period |
| RSI | Relative Strength Index |
| SMA | Simple Moving Average |
| SQL | Structured Query Language |
| UI | User Interface |
| VWAP | Volume-Weighted Average Price |
| WebHDFS | HTTP REST interface to HDFS |

# 1. Introduction

## 1.1 Background

The Nepal Stock Exchange (NEPSE) is the only stock exchange in Nepal. Trading takes place on working days between 11:00 and 15:00 Nepal Time (NPT). The official company list used in this project contains 649 listed securities — 489 ordinary shares, 89 debentures, 63 mutual funds and 8 preference shares — across sectors such as commercial banks, hydropower, development banks, microfinance, insurance and manufacturing.

Market data is published by the exchange itself and by financial portals such as ShareSansar and MeroLagani. These sites show the current price of each stock, but an investor who wants to know whether a price move is unusual, how a sector has performed over the last month, or how today compares with the last ten years has to collect and combine the data manually.

## 1.2 Motivation

Stock market data is a natural big data problem: during trading hours hundreds of symbols change price many times per minute (velocity), the history spans decades (volume), and the data comes from several sources in different formats with real quality problems (variety and veracity). A single script and a relational database struggle to ingest a live stream, analyse half a million historical rows and serve the results at the same time. A distributed stack built from Kafka, Spark and HDFS handles each of these jobs with a tool designed for it.

## 1.3 Objectives

- Collect real-time and historical NEPSE data from public sources.
- Ingest live (or replayed) price ticks through **Apache Kafka**.
- Process the stream in real time with **Spark Structured Streaming**: candles, market snapshot, sector flow and alerts.
- Process 31 years of history with **Spark batch / Spark SQL**: cleaning and technical indicators.
- Store raw, cleaned and analytical data in an **HDFS** data lake using Parquet.
- Present everything on a dashboard that works whether or not the market is open.
- Demonstrate fault tolerance and, as a bonus, a distributed machine-learning model (Spark MLlib).

## 1.4 Scope

The system covers all securities traded on NEPSE, daily price history from 1995 (dense from 2011) to the present, and intraday ticks for live or replayed sessions. It runs on a single machine with Docker; every component is a real distributed service (Kafka broker, HDFS NameNode and DataNode, Spark master and worker) and would scale out by adding nodes. The project is an analytics tool, not a trading system, and nothing in it is investment advice.

# 2. Problem Definition

## 2.1 Problem statement

**Retail investors in Nepal have no single, free tool that combines real-time NEPSE market monitoring with historical context.** In particular they cannot easily:

- see live prices, turnover and the day's biggest movers for the whole market in one place;
- get notified automatically about unusual activity — volume spikes, sudden price moves, or stocks hitting the ±10 % daily circuit limit;
- compare today's behaviour with years of history through technical indicators (moving averages, RSI, 52-week range) and sector trends;
- trust the historical numbers — raw data contains inconsistent rows and price drops caused by bonus and right shares that look like crashes.

The goal of this project is to design and build a big data pipeline that ingests NEPSE data **live or as a replay of any past day**, processes it **in real time and in batch**, stores it in a **data lake**, and presents insights and alerts on a dashboard.

## 2.2 Why this is a big data problem

**Table 1:** The five Vs of big data applied to NEPSE

| Characteristic | How it appears in NEPSE data |
|---|---|
| Volume | 514,018 daily price records for 372 companies (1995–2026), 6,196 days of NEPSE index; a single replayed trading day produces 15,142 intraday ticks for 327 symbols, and a live day produces more. |
| Velocity | Live prices for ~350 symbols polled every 5 seconds during trading hours; alerts must be raised within seconds. |
| Variety | JSON from the NEPSE API, HTML tables from ShareSansar, CSV files from GitHub, TradingView-style JSON from MeroLagani, reference data (company list) and recorded sessions (JSONL). |
| Veracity | 40,552 rows (8 %) with a high/low that does not contain the open/close, 20 invalid rows, and 731 artificial price drops from bonus/right-share adjustments. |
| Value | Market overview, unusual-activity alerts, technical analysis and sector money flow that are not available in one place today. |

# 3. Data Sources

All sources are public. Each was tested during the project; Table 2 lists what is collected from each and how.

**Table 2:** Data sources

| # | Data | Source | Type / format | Collected by |
|---|---|---|---|---|
| 1 | Live market: last traded price, open/high/low, volume, turnover, last trade quantity, % change | nepalstock.com official API (via the open-source NepseUnofficialApi library) | Real-time · JSON | `live_producer.py` (polls every 5 s) |
| 2 | Live market (fallback) | ShareSansar live-trading page | Real-time · HTML table | `live_producer.py` (used automatically if the API fails) |
| 3 | Daily OHLCV + turnover for ~370 companies, 1995 → today | GitHub dataset Aabishkar2/nepse-data (updated daily from the exchange) | Batch · CSV | `fetch_history.py` |
| 4 | NEPSE index history since 2000; per-symbol history (fallback) | MeroLagani chart API | Batch · JSON | `fetch_history.py` |
| 5 | Company master: symbol, name, sector, instrument type (649 securities) | nepalstock.com official API | Reference · JSON → CSV | `fetch_companies.py` |
| 6 | Recorded live sessions | Our own live producer, saved during market hours | Replay · JSONL | `live_producer.py` |

A small **offline sample** (42 stocks — the three most traded in every sector plus well-known blue chips — from 2019 to today, 4 MB) is committed to the project repository so the complete system can run without internet access.

## 3.1 Common message format

Both live and replay producers publish the same JSON message to Kafka, keyed by stock symbol. Because the format is identical, the Spark jobs do not need to know which mode is running:

{"symbol": "NABIL", "ts": "2026-10-09T14:59:59+05:45", "ltp": 536.2,
 "open": 532.0, "high": 538.0, "low": 531.0, "prev_close": 533.0,
 "volume": 48779, "turnover": 26163739.6, "last_qty": 10, "change_pct": 0.6,
 "source": "nepse-api", "mode": "LIVE",
 "session_start": "2026-10-09T11:00:00+05:45", "session_date": "2026-10-09"}

# 4. Important Features Considered

## 4.1 Data features (raw)

**Table 3:** Raw data features

| Feature | Meaning |
|---|---|
| symbol, company_name, sector, instrument_type | Identity and classification of the security (from the company master) |
| date / ts | Trading day (history) or event time of a tick (stream), Nepal time |
| open, high, low, close / ltp | Prices; ltp = last traded price |
| prev_close, change_pct | Previous day's close and percentage change — basis for the ±10 % circuit rule |
| volume, turnover | Shares traded and value traded in NPR (cumulative for the day in ticks) |
| last_qty | Quantity of the most recent trade — used for per-minute volume and VWAP |

## 4.2 Derived analytical features

**Table 4:** Derived features

| Feature | Layer | Purpose |
|---|---|---|
| 1-minute OHLC candles, VWAP | Streaming | Intraday price chart; volume-weighted average price |
| Live state per symbol, gainers/losers, most active | Streaming | Market overview |
| Sector money flow per minute | Streaming | Which sectors are attracting money right now |
| Circuit alert (\|change\| ≥ 9.5 %) | Streaming | Stocks at or near NEPSE's ±10 % daily limit |
| Volume-spike alert (volume ≥ 3 × 20-day average) | Streaming + batch | Unusual activity (stream joined with batch baseline) |
| Big-move alert (≥ 5 % from today's open) | Streaming | Sudden intraday swings |
| SMA 20 / 50 / 200, Bollinger bands, RSI 14 | Batch | Trend and momentum |
| 52-week high/low, 1w/1m/3m/1y returns, volatility | Batch | Long-term context and risk |
| Golden / death cross, trend label, RSI signal | Batch | Simple, explainable signals for the screener |

| Feature | Layer | Purpose |
|---|---|---|
| Sector and market daily aggregates, breadth | Batch | Sector rotation and market health |
| Corporate-action adjusted prices (adj_*) | Batch | Remove artificial drops caused by bonus/right shares |
| Next-day up/down probability | Batch (MLlib) | Bonus: machine-learning demonstration |

## 4.3 System design features

- **Scalability** — Kafka partitions the tick topic by symbol; Spark processes partitions in parallel; HDFS distributes blocks over DataNodes.
- **Fault tolerance** — Kafka retains messages for 7 days; Spark Structured Streaming checkpoints offsets and state in HDFS, so a crashed job resumes exactly where it stopped.
- **Low latency** — alerts and the live snapshot are updated every 5 seconds; 1-minute candles are written about 10–20 seconds after the minute closes.
- **Data quality** — validation, de-duplication, OHLC repair and corporate-action adjustment in the batch layer, with a stored data-quality report.
- **Source independence** — LIVE, REPLAY and HISTORICAL modes share one message schema and one pipeline, so the demo never depends on market hours.
- **Efficient storage** — columnar Parquet, sensible partitioning, and a log-compacted Kafka topic for the latest state.

# 5. System Design

## 5.1 Choice of big data components

The assignment asks for two big data tools. This project uses **Apache Kafka** and **Apache Spark** as its two main tools, with **Hadoop HDFS** as the storage layer that both of them use.

### Apache Kafka — ingestion and messaging

Kafka is a distributed, partitioned commit log. Producers append messages to topics and consumers read them at their own pace using offsets. It was chosen because it **decouples** the data sources from processing (if Spark is restarted, ticks wait safely in Kafka), **absorbs bursts** at market open, keeps data for **replay**, and lets the live and replay producers write to the same topic. Kafka also serves processed results to the dashboard: the `nepse.analytics` topic is log-compacted, so it keeps only the latest message per symbol.

### Apache Spark — stream and batch processing

Spark offers one engine and one DataFrame API for both **Structured Streaming** (micro-batch stream processing with event-time windows, watermarks, stateful aggregation and exactly-once file sinks) and **batch** processing with Spark SQL and window functions. The same team can therefore write the speed layer and the batch layer with the same skills. Spark MLlib adds distributed machine learning on the same data.

### Hadoop HDFS — storage

HDFS stores large files as blocks spread over DataNodes, with the NameNode keeping the metadata. It suits write-once, read-many analytical data, integrates natively with Spark, and stores Spark's streaming checkpoints. Data is kept as **Parquet**, a compressed columnar format: Spark reads only the columns a query needs.

**Table 5:** Big data components chosen and alternatives

| Need | Chosen | Alternatives considered | Reason for the choice |
|---|---|---|---|
| Ingestion / messaging | Kafka | Apache Flume, RabbitMQ | Flume moves data into Hadoop but cannot replay to many consumers; RabbitMQ deletes messages once consumed. Kafka keeps an ordered, replayable log. |

| Need | Chosen | Alternatives considered | Reason for the choice |
|---|---|---|---|
| Stream processing | Spark Structured Streaming | Apache Flink, Kafka Streams | Flink has lower latency, but Spark gives one API for streaming, batch, SQL and ML; 5-second micro-batches are fast enough for NEPSE. |
| Batch processing | Spark SQL | Hadoop MapReduce, Hive | Spark runs window-heavy indicator jobs in memory, far faster than MapReduce, and shares code with the stream. |
| Storage | HDFS + Parquet | Cassandra, HBase, MongoDB | Analytics scan whole columns over years of data — a columnar data lake fits better than key-value stores, which suit single-row lookups. |
| Serving latest state | Compacted Kafka topic | Redis, Cassandra | Avoids another database: compaction keeps only the newest message per symbol. |

## 5.2 Block diagram

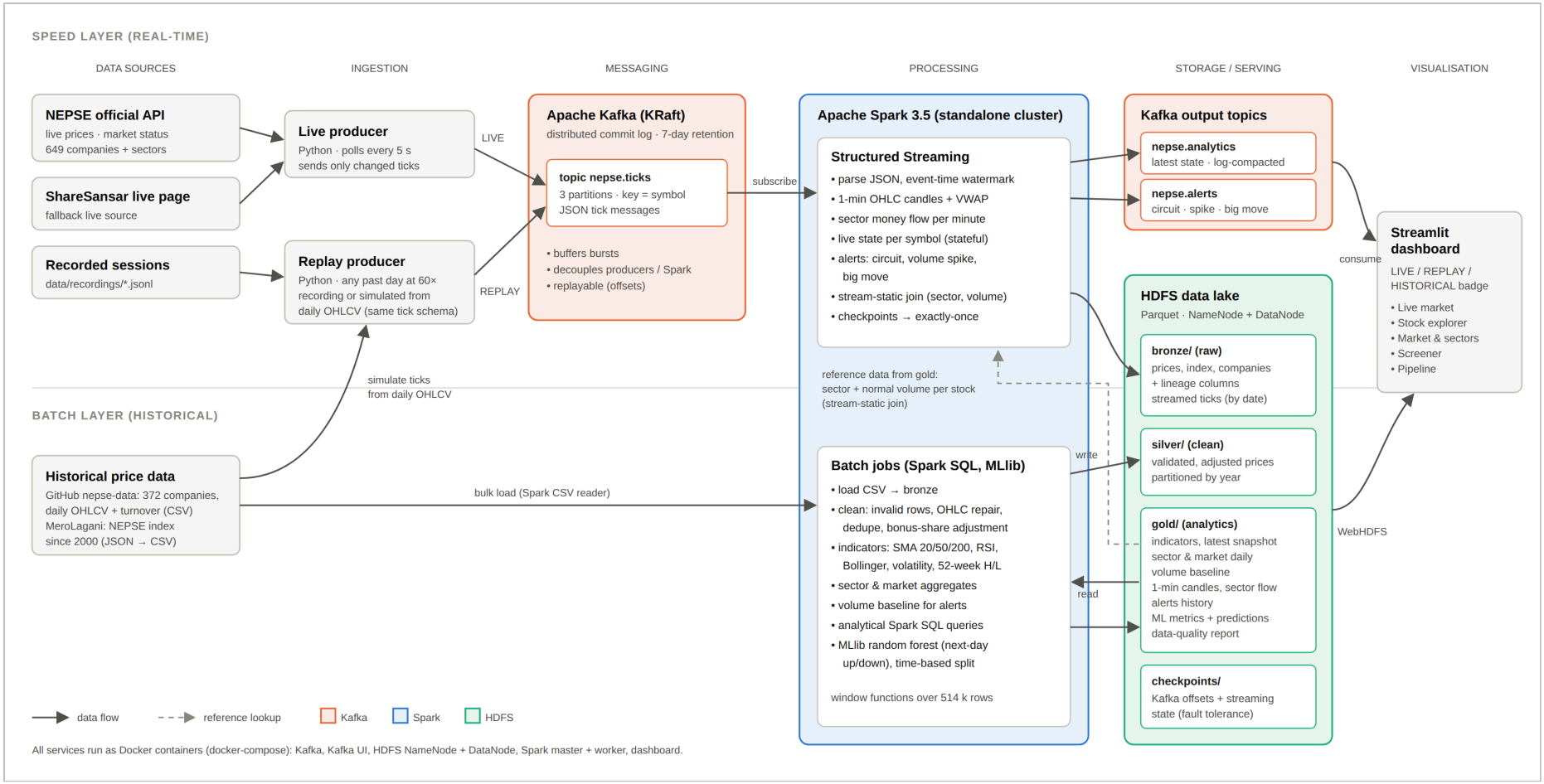



**Figure 1:** Block diagram of the NEPSE big data system (speed layer above, batch layer below)

Data flows through the system as follows (Figure 1):

1. **Sources → ingestion.** The live producer polls the NEPSE API every 5 seconds (ShareSansar as fallback) and publishes only ticks whose price or volume changed. The replay producer streams a recorded session, or simulates intraday ticks from a day's open/high/low/close/volume, at a chosen speed.
2. **Kafka.** Ticks land in `nepse.ticks` (3 partitions, keyed by symbol, so each stock's ticks stay in order).
3. **Spark Structured Streaming** reads the topic, applies an event-time watermark, and runs five queries: raw ticks → HDFS bronze; 1-minute candles and sector flow → HDFS gold; live state per symbol → Kafka `nepse.analytics`; alerts → Kafka `nepse.alerts` and HDFS gold.
4. **Batch layer.** Spark loads the historical CSV files into HDFS bronze, cleans them into silver, computes indicators and aggregates into gold, trains the MLlib model and answers analytical questions with Spark SQL.
5. **Stream-static join.** The streaming job enriches every tick with batch results (sector, normal 20-day volume), which connects the two layers.
6. **Dashboard.** Streamlit consumes the two Kafka output topics for the live view and reads gold Parquet files from HDFS (WebHDFS) for history.

## 5.3 Data storage design

### 5.3.1 HDFS data lake: bronze, silver and gold layers

The data lake follows the common bronze / silver / gold (medallion) layout. Each layer has a single job, and any layer can be rebuilt from the one below it.

**Table 6:** HDFS data lake layout

| Layer | HDFS path | Content | Partitioning | Size |
|---|---|---|---|---|
| Bronze (raw) | /nepse/bronze/prices, index, companies | CSV data exactly as received + lineage columns (_source_file, _ingested_at) | — | 11.9 MB |
| Bronze (raw) | /nepse/bronze/ticks | Every streamed tick | trade_date | grows per session |

| Layer | HDFS path | Content | Partitioning | Size |
|---|---|---|---|---|
| Silver (clean) | /nepse/silver/prices, index, companies | Validated, de-duplicated, OHLC-repaired, bonus-adjusted prices with sector | year | 23.1 MB |
| Gold (analytics) | /nepse/gold/indicators | SMA, RSI, Bollinger, volatility, returns, signals per symbol per day | year | 51.7 MB |
| Gold (analytics) | /nepse/gold/latest_snapshot, volume_baseline, sector_daily, market_daily, data_quality, ml_* | Screener, alert baseline, sector/market history, quality report, ML results | year (sector_daily) | ≈ 1.5 MB |
| Gold (streaming) | /nepse/gold/candles_1m, sector_flow_1m, alerts | Outputs of Spark Structured Streaming | trade_date | grows per session |
| Checkpoints | /nepse/checkpoints/<query> | Kafka offsets and streaming state for exactly-once recovery | — | small |

### 5.3.2 File format: Parquet

All HDFS data is stored as Parquet. Being columnar, a query that needs only `date` and `adj_close` reads only those columns, and similar values in a column compress well. The 33 MB of raw price CSV files occupy **11.7 MB** as Parquet in the bronze layer — about **2.8 × smaller** — even after adding two lineage columns.

### 5.3.3 Partitioning strategy

Historical tables are partitioned by **year**, not by day. Partitioning by day would create about 3,500 directories of tiny files — the well-known HDFS **small-files problem**, where every file costs NameNode memory and adds task overhead. Yearly partitions still let queries such as “last 5 years” skip old data. Streaming outputs are partitioned by `trade_date`, which suits their append-only, per-session nature.

### 5.3.4 Kafka topics

**Table 7:** Kafka topics

| Topic | Partitions | Key | Retention / cleanup | Content |
|---|---|---|---|---|
| nepse.ticks | 3 | symbol | 7 days (delete) | Raw ticks from the live or replay producer |

| Topic | Partitions | Key | Retention / cleanup | Content |
|---|---|---|---|---|
| nepse.analytics | 1 | symbol | log compaction | Latest state per symbol computed by Spark |
| nepse.alerts | 1 | symbol | 7 days (delete) | Circuit, volume-spike and big-move alerts |

Log compaction on `nepse.analytics` made a measurable difference: after many replays the topic shrank from 23 MB to 1.5 MB and the dashboard's start-up catch-up time fell from **19.2 s to 1.4 s**.

### 5.3.5 Replication

HDFS normally keeps three copies of every block on different DataNodes. This single-machine deployment has one DataNode, so the replication factor is set to 1 (for both the NameNode configuration and the Spark HDFS client). Moving to a real cluster only requires more DataNodes and raising `dfs.replication` to 3; no code changes.

## 5.4 Operating modes

NEPSE trades only about four hours per working day, and the demonstration may take place when the market is closed. The system therefore supports three modes that share one pipeline:

**Table 8:** Operating modes

| Mode | When used | Data path | Dashboard badge |
|---|---|---|---|
| LIVE | Market open | NEPSE API → live producer → Kafka → Spark Streaming | red “LIVE” |
| REPLAY | Market closed | Recording or historical day → replay producer (e.g. 60×) → Kafka → Spark Streaming | orange “REPLAY · replaying the 2026-10-09 session” |
| HISTORICAL | Always | History CSV → Spark batch → HDFS gold | blue “HISTORICAL” (no active stream) |

In REPLAY mode the timestamps are shifted to the current time and compressed by the replay speed, so Spark sees a fresh, live-looking stream every time. Each replay loop is tagged as a separate **session**, so results from two loops are never mixed.

# 6. Implementation

## 6.1 Environment and technology stack

**Table 9:** Technology stack

| Component | Version | Role |
|---|---|---|
| Docker / Docker Compose | 29.1 / 5.4 | Runs every service as a container on one machine |
| Apache Kafka (KRaft mode, no ZooKeeper) | 3.9.0 | Message broker |
| Kafka UI (provectuslabs) | 0.7.2 | Topic and message browser for the demo |
| Apache Hadoop HDFS (NameNode + DataNode) | 3.3.6 | Distributed file system / data lake |
| Apache Spark (master + worker, PySpark) | 3.5.3 | Structured Streaming, Spark SQL, MLlib |
| Python + uv (ingestion container) | 3.12 / 0.4 | Data collection, live and replay producers |
| Streamlit + Plotly | 1.45 / 5.24 | Dashboard |

The Spark image has the Kafka connector JARs built in, so jobs start without downloading anything and the whole stack works offline once the images are built. Each Spark application may use at most two of the worker's four cores, so the streaming job and a batch job can run at the same time.

## 6.2 Data collection and ingestion

- `fetch_companies.py` downloads the 649-security company list with sectors from the NEPSE API.
- `fetch_history.py` downloads daily OHLCV and turnover for all companies (GitHub dataset) and the NEPSE index (MeroLagani), and normalises them into one CSV schema.
- `live_producer.py` checks the NEPSE market-status API, polls live prices every 5 seconds, sends only changed ticks to Kafka and records the session to a JSONL file for later replay.
- `replay_producer.py` streams a recorded session, or simulates a session from daily data: the simulated price path starts at the day's open, touches its high and low and ends exactly at its close, with the day's volume split across trades. The

NABIL replay for 2026-10-09 ends at the real close (536.2), high (538), low (531) and volume (48,779); simulated turnover is within 0.4 % of the real value.

## 6.3 Batch processing (Spark SQL)

`batch_load_history.py` loads the CSV files into bronze and builds silver with these rules:

1. Drop rows with missing or non-positive prices or zero volume (20 rows).
2. Drop duplicate (symbol, date) rows (none found).
3. Repair OHLC: widen high/low to include open and close (40,552 rows).
4. Detect corporate actions: NEPSE's daily circuit is ±10 %, so an opening gap below −10.5 % must be a bonus or right-share adjustment rather than trading. Earlier prices are multiplied by the gap ratio (back-adjustment) into `adj_*` columns, using a window sum of log-ratios over later rows (731 adjustments).
5. Recompute change % from the previous close and attach sector and instrument type.

`batch_indicators.py` then computes all indicators with Spark window functions partitioned by symbol and ordered by date, and writes the gold tables. Sector and market statistics count ordinary shares only, so debentures and mutual funds do not distort them. `batch_queries.py` answers analytical questions in Spark SQL, for example:

```
SELECT sector, ROUND(SUM(turnover) / 1e9, 2) AS turnover_billion_npr,
       SUM(advancers) AS advances, SUM(decliners) AS declines
FROM sector_daily
WHERE date >= date_sub((SELECT MAX(date) FROM sector_daily), 30)
GROUP BY sector ORDER BY turnover_billion_npr DESC
```

## 6.4 Stream processing (Spark Structured Streaming)

**Table 10:** Streaming queries

| Query | Output | Technique |
|---|---|---|
| ticks | HDFS bronze/ticks | Exactly-once Parquet file sink, partitioned by trade_date |
| candles_1m | HDFS gold/candles_1m | 1-minute event-time tumbling windows, 10 s watermark, OHLC + VWAP |
| sector_flow | HDFS gold/sector_flow_1m | Windowed aggregation by sector after a stream-static join |

| Query | Output | Technique |
|---|---|---|
| live_snapshot | Kafka nepse.analytics | Stateful aggregation (latest tick per symbol per session), update output mode |
| alerts | Kafka nepse.alerts + HDFS gold/alerts | Filters on enriched ticks, de-duplicated with dropDuplicatesWithinWatermark, written by foreachBatch to both sinks |

All five queries run in one Spark application, each with its own checkpoint directory in HDFS. Ticks are joined with two batch-layer tables — the company list (sector) and the 20-day volume baseline — so the volume-spike alert compares today's volume with the stock's normal volume.

## 6.5 Machine learning (Spark MLlib, bonus)

`ml_predict.py` builds ten features from the gold indicators — daily return, 1-week and 1-month return, RSI, distance from SMA 20 and SMA 50, SMA 50 vs SMA 200, position within the Bollinger band, volume ratio and volatility — and a label that is 1 if the stock closes higher the next day. A random forest (60 trees, depth 7) is trained on 2015–2024 and tested on 2025 onward. This **time-based split** prevents the model from learning from the future, a common mistake with random splits on time series.

## 6.6 Dashboard

The Streamlit dashboard has five tabs. The live tab refreshes every 3 seconds; the others read the gold layer from HDFS.

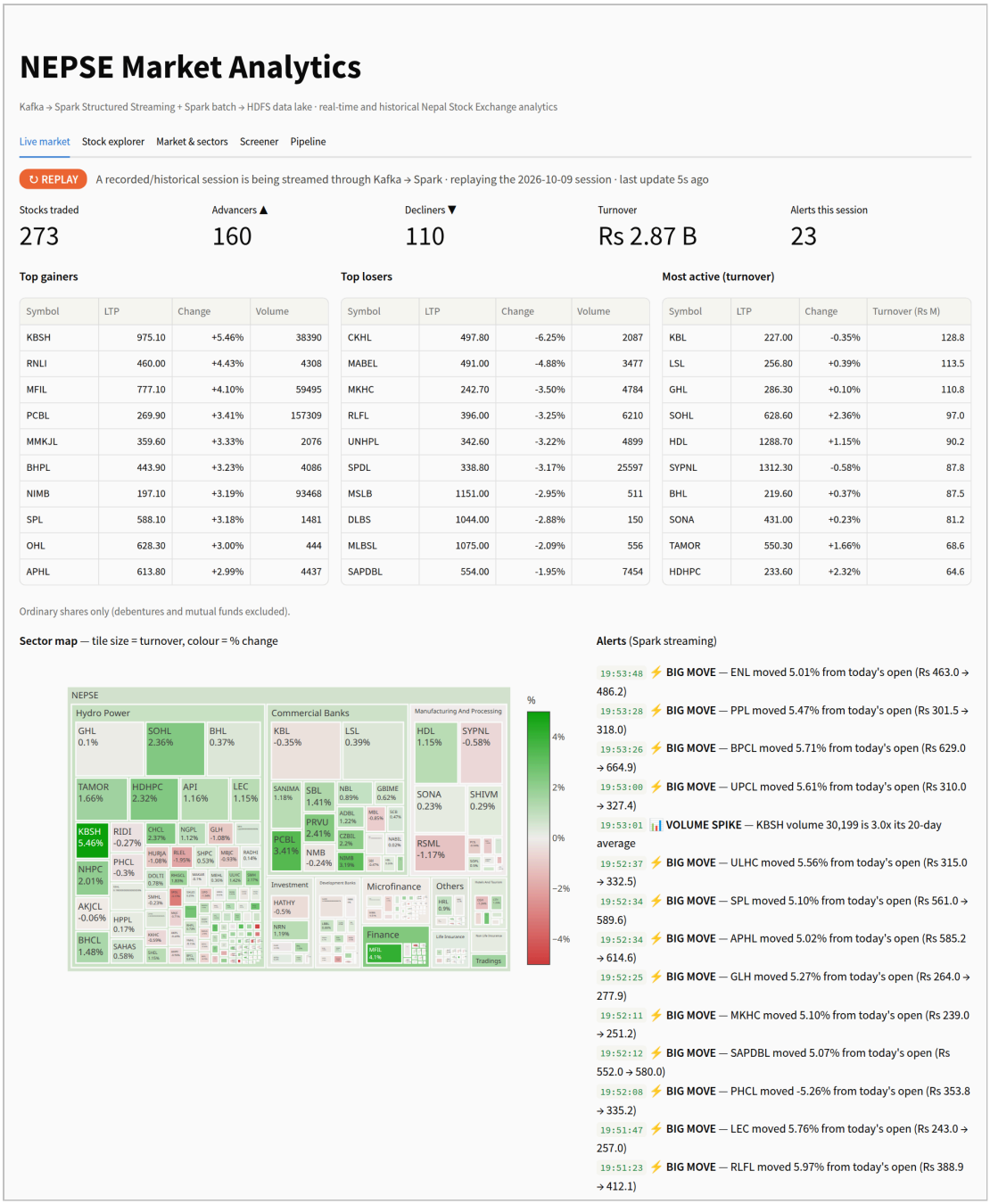



**Figure 2:** Live market tab during a REPLAY session: mode badge, market KPIs, movers, sector map and Spark alerts




**Figure 3:** Stock explorer: bonus-adjusted candlesticks with SMA 20/50/200, Bollinger band, volume and RSI

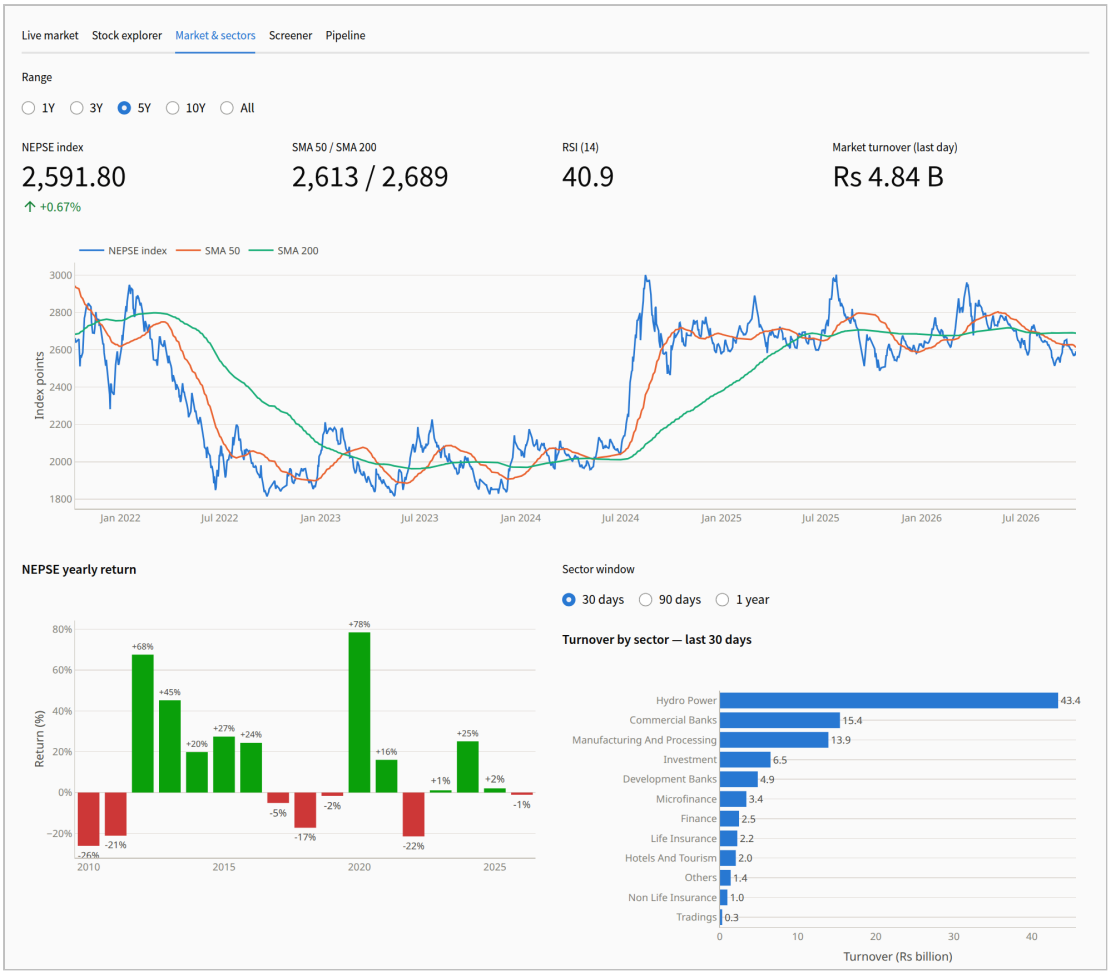
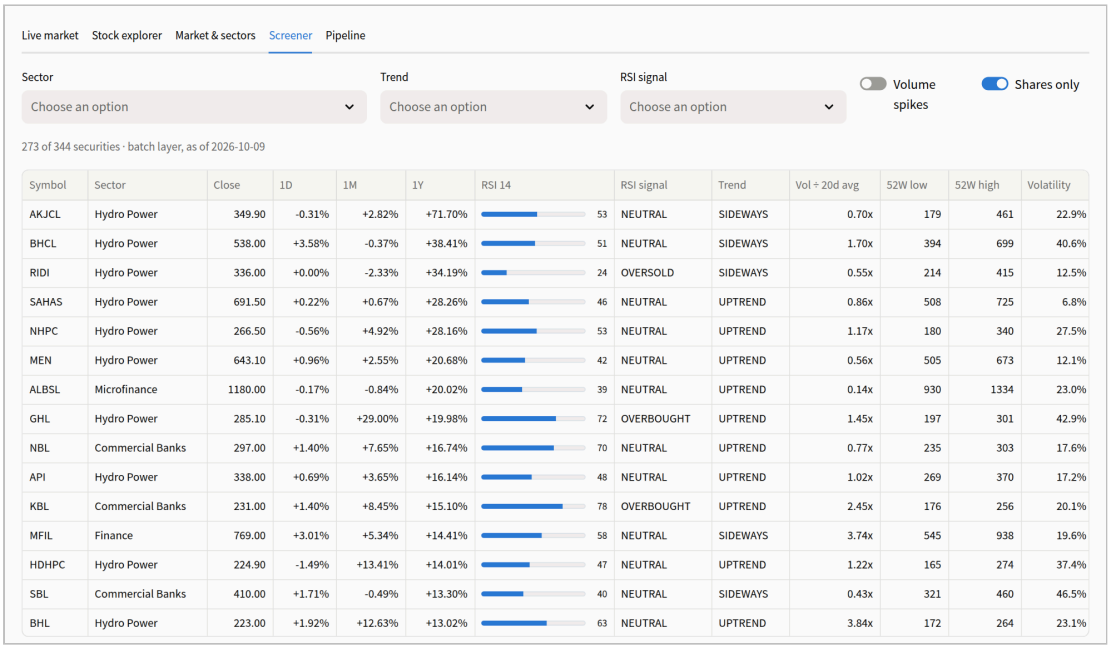



**Figure 4:** Market & sectors tab: NEPSE index with moving averages, yearly returns and turnover by sector




| Symbol | Sector | Close | 1D | 1M | 1Y | RSI 14 | RSI signal | Trend | Vol ÷ 20d avg | 52W low | 52W high | Volatility |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| AKJCL | Hydro Power | 349.90 | -0.31% | +2.82% | +71.70% | 53 | NEUTRAL | SIDEWAYS | 0.70x | 179 | 461 | 22.9% |
| BHCL | Hydro Power | 538.00 | +3.58% | -0.37% | +38.41% | 51 | NEUTRAL | SIDEWAYS | 1.70x | 394 | 699 | 40.6% |
| RIDI | Hydro Power | 336.00 | +0.00% | -2.33% | +34.19% | 24 | OVERSOLD | SIDEWAYS | 0.55x | 214 | 415 | 12.5% |
| SAHAS | Hydro Power | 691.50 | +0.22% | +0.67% | +28.26% | 46 | NEUTRAL | UPTREND | 0.86x | 508 | 725 | 6.8% |
| NHPC | Hydro Power | 266.50 | -0.56% | +4.92% | +28.16% | 53 | NEUTRAL | UPTREND | 1.17x | 180 | 340 | 27.5% |
| MEN | Hydro Power | 643.10 | +0.96% | +2.55% | +20.68% | 42 | NEUTRAL | UPTREND | 0.56x | 505 | 673 | 12.1% |
| ALBSL | Microfinance | 1180.00 | -0.17% | -0.84% | +20.02% | 39 | NEUTRAL | UPTREND | 0.14x | 930 | 1334 | 23.0% |
| GHL | Hydro Power | 285.10 | -0.31% | +29.00% | +19.98% | 72 | OVERBOUGHT | UPTREND | 1.45x | 197 | 301 | 42.9% |
| NBL | Commercial Banks | 297.00 | +1.40% | +7.65% | +16.74% | 70 | NEUTRAL | UPTREND | 0.77x | 235 | 303 | 17.6% |
| API | Hydro Power | 338.00 | +0.69% | +3.65% | +16.14% | 48 | NEUTRAL | UPTREND | 1.02x | 269 | 370 | 17.2% |
| KBL | Commercial Banks | 231.00 | +1.40% | +8.45% | +15.10% | 78 | OVERBOUGHT | UPTREND | 2.45x | 176 | 256 | 20.1% |
| MFIL | Finance | 769.00 | +3.01% | +5.34% | +14.41% | 58 | NEUTRAL | SIDEWAYS | 3.74x | 545 | 938 | 19.6% |
| HDHPC | Hydro Power | 224.90 | -1.49% | +13.41% | +14.01% | 47 | NEUTRAL | UPTREND | 1.22x | 165 | 274 | 37.4% |
| SBL | Commercial Banks | 410.00 | +1.71% | -0.49% | +13.30% | 40 | NEUTRAL | SIDEWAYS | 0.43x | 321 | 460 | 46.5% |
| BHL | Hydro Power | 223.00 | +1.92% | +12.63% | +13.02% | 63 | NEUTRAL | UPTREND | 3.84x | 172 | 264 | 23.1% |

**Figure 5:** Screener: all securities filtered by sector, trend, RSI signal and volume spike

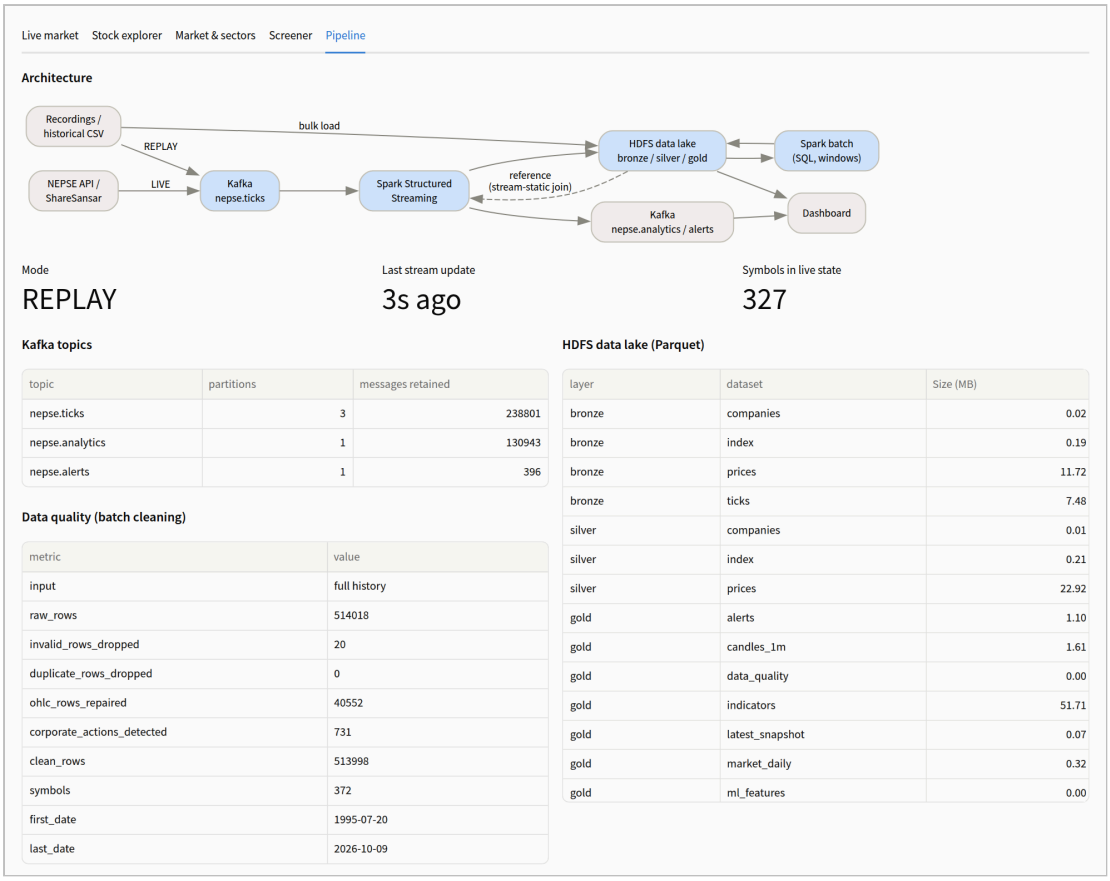



**Figure 6:** Pipeline tab: architecture, Kafka topic sizes, data-quality report and HDFS datasets

# 7. Results and Evaluation

## 7.1 Data quality

**Table 11:** Data-quality report produced by the batch layer

| Metric | Value |
|---|---|
| Raw price rows loaded | 514,018 |
| Invalid rows dropped | 20 |
| Duplicate rows dropped | 0 |
| Rows with OHLC repaired | 40,552 (7.9 %) |
| Bonus/right-share adjustments detected | 731 |
| Clean rows in silver | 513,998 |
| Companies | 372 |
| Date range | 1995-07-20 → 2026-10-09 |

The corporate-action detection is easy to verify on NABIL Bank: it found ten adjustments, including 2017-09-12, when the price opened at 1,254 after closing at 1,630 — a ratio of 0.769, i.e. a 30 % bonus share. Without the adjustment, moving averages and 52-week ranges would treat these days as crashes.

## 7.2 Sample analytical results

**Table 12:** NEPSE yearly returns computed with Spark SQL (selected years)

| Year | 2012 | 2013 | 2016 | 2017 | 2018 | 2020 | 2021 | 2022 | 2024 | 2025 |
|---|---|---|---|---|---|---|---|---|---|---|
| NEPSE return (%) | +67.6 | +45.2 | +24.3 | −5.1 | −17.3 | +78.5 | +16.1 | −21.5 | +25.1 | +2.0 |

Over the last 30 trading days, hydropower was by far the most traded sector (NPR 43.4 billion), followed by commercial banks (15.4 billion) and manufacturing (13.9 billion). On the latest trading day (2026-10-09) the screener flagged NABIL as oversold (RSI 29.7) and found six ordinary shares trading at three or more times their normal volume.

## 7.3 Streaming results

- A full trading day (2026-10-09) replayed at 60× produced **15,142 ticks for 327 symbols** in about 4 minutes, all received by Kafka and stored in HDFS.
- Each replayed session raised about 30 alerts — mostly big intraday moves (e.g. MABEL +7.7 % from the open) and volume spikes (KBSH, CGH, BHL at 3×

their normal volume).

- The final live state of each stock matched the real end-of-day data; for NABIL: 536.2, +0.6 %, 48,779 shares, VWAP 534.19.
- Latency: the live snapshot and alerts refresh every 5 seconds; a 1-minute candle is written 10–20 seconds after its minute closes.

## 7.4 Fault-tolerance test

A full day was replayed while the streaming job was deliberately stopped about one minute into the session and restarted 50 seconds later; ticks kept arriving in Kafka while Spark was down. Afterwards HDFS contained **15,142 ticks, all distinct, out of 15,142 sent** — nothing lost and nothing duplicated. This works because Spark stores its Kafka offsets in the checkpoint and the Parquet file sink commits each micro-batch exactly once. The 1-minute candles were continuous across the outage. (Alerts use `foreachBatch`, which is at-least-once; a duplicate alert after a crash is acceptable.)

## 7.5 Performance and resources

**Table 13:** Performance measurements

| Measurement | Result |
|---|---|
| Load + clean 514 k rows (CSV → bronze → silver) | ≈ 1.5 min |
| Indicators and gold tables | ≈ 1.7 min |
| MLlib training and evaluation (351 k labelled rows) | ≈ 2 min |
| Whole batch layer incl. SQL showcase | ≈ 5.5 min |
| Compression: raw CSV → Parquet (bronze) | 33 MB → 11.7 MB |
| Dashboard start-up catch-up (after log compaction) | 19.2 s → 1.4 s |
| Memory used by the stack when idle | ≈ 1.7 GB |
| Hardware | 8 CPU cores, 11 GB RAM laptop; Spark worker: 4 cores, 2 GB |

## 7.6 Machine-learning results

**Table 14:** Spark MLlib random forest — next-day direction

| Metric | Value |
|---|---:|
| Training rows (2015–2024) | 255,557 |
| Test rows (2025 → 2026-10) | 95,838 |
| Test accuracy | 55.8 % |
| Baseline: always predict the majority class (“down”) | 55.9 % |

| Metric | Value |
|---|---|
| AUC | 0.519 |
| Accuracy of confident predictions (p ≥ 0.6 or ≤ 0.4; 965 rows) | 64 % |

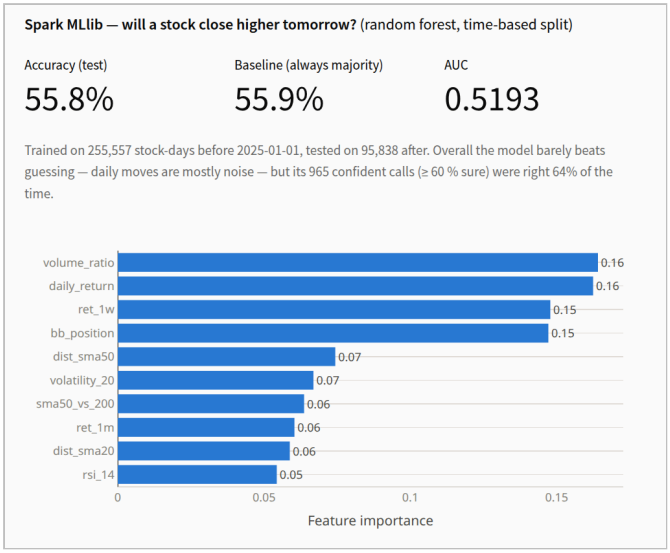



**Figure 7:** MLlib results and feature importance shown on the dashboard

Overall the model is no better than always guessing “down”, and its AUC is only slightly above 0.5. This is the expected and honest outcome: daily price changes are close to random, and publicly available technical indicators carry little predictive information. The model's rare confident calls were right 64 % of the time, which hints at a weak signal, but on too few cases to act on. The most useful features were volume ratio, the latest daily return, the 1-week return and the Bollinger-band position. The value of this part is to show a distributed, leakage-free ML pipeline, not to predict prices.

## 7.7 Problems encountered and solutions

**Table 15:** Problems encountered and how they were solved

| Problem | Solution |
|---|---|
| The demo may happen when NEPSE is closed | REPLAY mode streams any past day through the same pipeline |
| Bonus/right shares look like 20–30 % crashes | Detect gaps beyond the ±10 % circuit and back-adjust prices |
| Debentures and mutual funds appeared as top-gaining “bank” stocks | Filter statistics to ordinary shares using the instrument type |
| Spark wrote HDFS files with 3 replicas on a 1-DataNode cluster | Set the HDFS client replication to 1 in Spark's configuration |

| Problem | Solution |
|---|---|
| Streaming job took all 4 worker cores; batch jobs waited forever | Cap every Spark application at 2 cores (spark.cores.max) |
| Replaying the same day twice would be dropped as late data | Replay shifts timestamps to now; each loop is a new session |
| Back-to-back replay loops suppressed each other's alerts | Include the session in the alert de-duplication key |
| Dashboard needed 19 s to read the analytics topic | Kafka log compaction on nepse.analytics (1.4 s) |
| Candles stored in UTC, compared with Nepal time | Convert timestamps explicitly in the dashboard |
| Forced image rebuild needs internet (would fail offline) | Start script reuses built images |

# 8. Demonstration Plan

The demo is run by one script, `./scripts/demo.sh`, which pauses between steps:

1. Start the stack (Kafka, HDFS, Spark, dashboard) and show the Kafka UI, HDFS and Spark web interfaces.
2. Batch layer: show the Spark SQL answers and the HDFS bronze/silver/gold folders; dashboard tabs Stock explorer, Market & sectors and Screener.
3. Start Spark Structured Streaming and show its five queries in the Spark UI.
4. Start the LIVE producer if NEPSE is open, otherwise REPLAY the latest trading day; show messages arriving in the Kafka UI and the Live market tab updating with alerts.
5. Fault tolerance: kill the streaming job, wait 20 seconds, restart it and show that it resumes without loss.
6. Pipeline tab: architecture, topic and dataset sizes, data quality and the MLlib result.

The demo works fully offline: the images are pre-built, the Spark image contains the Kafka connector, the dashboard loads no external resources (verified with all non-local network requests blocked) and the replay uses local data.

# 9. Limitations and Future Work

## 9.1 Limitations

- Runs on one machine: one Kafka broker, one DataNode (replication 1), one Spark worker. The design is distributed, but the deployment is not.
- Bonus shares smaller than about 10 % cannot be distinguished from normal price moves, so they are not adjusted; a few early records are also inconsistent.
- Simulated replay ticks follow the real daily open/high/low/close/volume but not the real intraday order of trades; recorded sessions solve this.
- RSI uses a simple moving average of gains and losses rather than Wilder's smoothing.
- The live source relies on an unofficial client for the NEPSE website, which may break if the website changes (ShareSansar is the fallback).
- The machine-learning model has no practical predictive power for daily moves.

## 9.2 Future work

- Deploy on a real multi-node cluster (3 Kafka brokers, 3 DataNodes with replication 3, several Spark workers).
- Use the official corporate-action (bonus/right/dividend) records for exact price adjustment.
- Add news and social-media sentiment (e.g. ShareSansar and MeroLagani headlines) as another stream and study its link with price moves.
- Compact small streaming output files periodically and register tables in a Hive metastore.
- Add user watchlists and push notifications for alerts on chosen stocks.

# 10. Conclusion

This project designed and implemented a working big data system for the Nepal Stock Exchange. **Apache Kafka** ingests live or replayed market ticks and serves processed results; **Apache Spark** processes the same data as a real-time stream (candles, market state, sector flow and alerts) and as a 31-year batch (cleaning, technical indicators, SQL analytics and an MLlib model); **HDFS** stores raw, clean and analytical data in Parquet in a bronze/silver/gold layout. Joining the batch outputs into the stream links the two layers, and the replay mode means the system can be demonstrated at any time.

The evaluation showed concrete results: half a million historical rows cleaned with real data-quality problems fixed, a full trading day streamed and analysed in minutes, exactly-once recovery after a crash, and an honest assessment of what machine learning can and cannot do with daily stock data. The project shows how components of the big data ecosystem combine to solve a real problem that is relevant to investors in Nepal.

# References


[1] Apache Software Foundation, “Apache Kafka Documentation.” https://kafka.apache.org/documentation/

[2] Apache Software Foundation, “Structured Streaming Programming Guide,” Apache Spark 3.5.3. https://spark.apache.org/docs/3.5.3/structured-streaming-programming-guide.html

[3] Apache Software Foundation, “Spark SQL, DataFrames and Datasets Guide,” Apache Spark 3.5.3. https://spark.apache.org/docs/3.5.3/sql-programming-guide.html

[4] Apache Software Foundation, “Machine Learning Library (MLlib) Guide,” Apache Spark 3.5.3. https://spark.apache.org/docs/3.5.3/ml-guide.html

[5] Apache Software Foundation, “HDFS Architecture,” Apache Hadoop 3.3.6. https://hadoop.apache.org/docs/r3.3.6/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[6] Apache Software Foundation, “Apache Parquet Documentation.” https://parquet.apache.org/docs/

[7] N. Marz and J. Warren, Big Data: Principles and Best Practices of Scalable Realtime Data Systems. Manning, 2015.

[8] Nepal Stock Exchange Ltd. https://www.nepalstock.com

[9] basic-bgnr, “NepseUnofficialApi,” GitHub. https://github.com/basic-bgnr/NepseUnofficialApi

[10] Aabishkar2, “nepse-data: Historical and current datasets of Nepal Stock Market listed companies,” GitHub. https://github.com/Aabishkar2/nepse-data

[11] MeroLagani. https://merolagani.com

[12] ShareSansar. https://www.sharesansar.com

[13] Streamlit Documentation. https://docs.streamlit.io

# Appendix A: Installation and Execution

All components of the system run as Docker containers managed with Docker Compose, including the Python data-collection and producer scripts, so no local Python installation is required. Table 16 lists the commands used to set up and operate the system, in order.

**Table 16:** Setup and execution commands

| Step | Command | Purpose |
|---|---|---|
| 1. Start services | `./scripts/start.sh` | Starts Kafka, HDFS, Spark and the dashboard; creates the Kafka topics and HDFS directories |
| 2. Verify | `./scripts/verify.sh` | Checks every service and runs an end-to-end Spark–Kafka–HDFS test |
| 3. Collect data | `./scripts/ingest.sh fetch_history` | Downloads daily price history and the NEPSE index (the company list is included in the repository) |
| 4. Batch processing | `./scripts/run_batch.sh` | Builds the bronze, silver and gold layers and trains the MLlib model |
| 5. Stream processing | `./scripts/stream.sh start` | Starts the Spark Structured Streaming job |
| 6. Data ingestion | `./scripts/ingest.sh live_producer` | Publishes live ticks to Kafka; `replay_producer` replays a past trading day instead |

**Table 17:** Web interfaces

| Service | Address |
|---|---|
| Dashboard | http://localhost:8501 |
| Kafka UI | http://localhost:8085 |
| HDFS NameNode | http://localhost:9870 |
| Spark master / streaming UI | http://localhost:8080 / http://localhost:4040 |

# Appendix B: Project Structure

```
docker-compose.yml      all services
config/hadoop.env       HDFS configuration
ingestion/              fetch_companies.py, fetch_history.py, build_sample.py,
                        live_producer.py, replay_producer.py, common.py, Dockerfile
spark/Dockerfile        Spark 3.5.3 image with the Kafka connector built in
spark/conf/             spark-defaults.conf, log4j2.properties
spark/jobs/             batch_load_history.py, batch_indicators.py, batch_queries.py,
                        ml_predict.py, stream_processor.py, stream_report.py
dashboard/              Streamlit app (app.py, data.py)
scripts/                start, stop, init, verify, run_batch, stream,
                        spark_submit, ingest, demo
data/                   sample/ (offline dataset), reference/,
                        historical/, recordings/
plan.md, README.md      design document and setup guide
```

# Appendix C: Division of Work

**Table 18:** Division of work

| **Santosh Tiwari (082MSDSA024) — ingestion and storage** | **Arbind Sah (082MSDSA006) — processing and visualisation** |
|---|---|
| Docker Compose setup (Kafka, HDFS) | Spark cluster configuration |
| Data collection scripts and data cleaning rules | Batch indicators and Spark SQL queries |
| Live producer and session recordings | Spark Structured Streaming job and alerts |
| Replay producer | Streamlit dashboard |
| HDFS data-lake layout (bronze/silver/gold) | MLlib model |
| Report: problem, sources and features (part a) | Report: storage, components and block diagram (part b) |